Reviewer Pack — Minimal Rank of Algebraic Curves Bounds AC⁰ Parity Circuit D…

Entry 91e077ae5181 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 23:40:33 UTC

Minimal Rank of Algebraic Curves Bounds AC⁰ Parity Circuit Depth

Entry ID: `91e077ae5181` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 23:40:33 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Curves) **Field B** (complexity object): Complexity Theory: Boolean Circuit Complexity

Statement:

> {'stmt1': 'For any polynomial-time computable function f(n), there exists a constant c_f such that for all n ≥ 1, the minimal rank of the algebraic curve corresponding to an AC⁰ circuit C computing PARITY on n inputs is at least c_f·log_2(f(n))□', 'stmt2': 'This holds regardless of the specific representation of the input and output in terms of field extensions or transcendental functions.', 'stmt3': 'For a single fixed instance of PARITY, this bound is asymptotically tight up to polynomial factors.'}

Rationale (proposer's reasoning):

> {'stmt1': 'Algebraic curves are known to encode complex computations in geometry, and their ranks can be computed efficiently for AC⁰ circuits.', 'stmt2': 'This conjecture bridges the gap between geometric invariants and circuit complexity by connecting them with PARITY, a fundamental problem in complexity theory.', 'stmt3': 'If true, this would provide a novel approach to proving lower bounds on PARITY, complementing existing techniques such as the switching lemma.'}

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `563195b2a6364b45`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of algebraic curves corresponding to AC⁰ circuits computing PARITY is at least c_f·log_2(f(n)) for all n ≥ 1, where f(n) is polynomial-time computable and c_f is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `algebraic geometry algebraic curves AND Boolean circuit complexity AND AC0 parity circuit depth` - `minimal rank algebraic curve OR algebraic geometry AND AC0 parity circuit complexity` - `circuit computing PARITY AND minimal rank bound algebraic geometry`

Top relevant hits considered: - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e}-linear maps - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/2211.06357v2] Parity of ranks of Jacobians of curves - [http://arxiv.org/abs/2310.08437v2] Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions - [http://arxiv.org/abs/1806.06290v1] Average-Case Lower Bounds and Satisfiability Algorithms for Small Threshold Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log2(x):
        if x <= 0:
            return float('-inf')
        return math.log2(x)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i+1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
```

```
            factor = Fraction(matrix[i][i])
            for j in range(cols):
                matrix[i][j] /= factor
            for j in range(rows):
                if i != j:
                    factor = Fraction(matrix[j][i])
                    for k in range(cols):
                        matrix[j][k] -= factor * matrix[i][k]
        rank = sum(1 for row in matrix if any(row))
        return rank

    def compute_minimal_rank(n):
        # Generate a random AC⁰ circuit for the PARITY function
        circuit = []
        for i in range(n):
            gate = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(gate)]
            circuit.append((gate, inputs))

        # Convert the circuit to an algebraic curve (simplified example)
        # This is a placeholder; actual computation would be more complex
        curve = []
        for gate, inputs in circuit:
            if gate == 'AND':
                curve.append(min(inputs))
            elif gate == 'OR':
                curve.append(max(inputs))

        # Compute the minimal rank of the algebraic curve
        matrix = [[curve[i] ** j for j in range(n+1)] for i in range(n+1)]
        return gaussian_elimination(matrix)

    n_values = [5, 10, 15, 20, 30, 40]
    ranks = []
    for n in n_values:
        rank = compute_minimal_rank(n)
        ranks.append(rank)

    mean_rank = sum(ranks) / len(ranks)
    std_rank = math.sqrt(sum((x - mean_rank) ** 2 for x in ranks) / len(ranks))

    conjecture_holds = all(rank >= n * log2(n) for rank, n in zip(ranks, n_values))
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean_rank,
        "instances_tested": len(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e5fdd80.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e5fdd80.py", line 69, in run_trial
    rank = compute_minimal_rank(n)
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4e5fdd80.py", line 50, in compute_minim
    inputs = [random.randint(0, 1) for _ in range(gate)]
                                           ^^^^^^^^^^^
TypeError: 'str' object cannot be interpreted as an integer
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the conjecture's validity according to the pre-registered support and falsification conditions. | next: Investigate the cause of the crash in the test code and attempt to run the test again to gather the necessary data for a proper evaluation.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12211
2	preregistration	ollama_remote	glm4:latest	0	0	6463
3	novelty	ollama_remote	glm4:latest	0	0	4705
4	novelty	ollama_remote	glm4:latest	0	0	6420
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16251
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7421
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8284
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17156
9	judge	ollama_remote	glm4:latest	0	0	14053

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 92964 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/91e077ae5181.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/91e077ae5181.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/91e077ae5181.tar.gz` (if generated)